

Les modules et les paquets Python

Table des matières

Importer un module.....	2
Les paquets.....	3
Composition.....	4
Héritage.....	5
Références.....	6

Document écrit par Stéphane Gill

Mai 2022



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

On appelle “module” tout fichier constitué de code Python (c’est-à-dire tout fichier avec l’extension .py) importé dans un autre fichier ou script.

Les modules permettent la séparation et donc une meilleure organisation du code. En effet, il est courant dans un projet de découper son code en différents fichiers qui vont contenir des parties cohérentes du programme final pour faciliter la compréhension générale du code, la maintenance et le travail d’équipe si on travaille à plusieurs sur le projet.

En Python, on peut distinguer trois grandes catégories de module en les classant selon leur éditeur :

- Les modules standards qui ne font pas partie du langage en soi mais sont intégrés automatiquement par Python ;
- Les modules développés par des développeurs externes qu’on va pouvoir utiliser ;
- Les modules développés par le programmeur lui-mêmes.

Un paquet est tout simplement un ensemble de plusieurs modules regroupés entre eux.

Importer un module

Un programme Python va généralement être composé d’un script principal qui va importer différents modules (c’est-à-dire différents fichiers Python) pour pouvoir les utiliser.

Pour importer un module, on utilise la syntaxe `import nom-de-mon-module`.

Exemple 1 : Importer un module

```
1  '''
2  Fichier : module1.py
3  '''
4
5  class module1():
6
7      def __init__(self, couleur):
8          self.couleur = couleur
9
10     def affiche(self):
11         print(self.couleur)
12
13 if __name__ == "__main__":
14     print ("Début du test")
15     a = module1(1)
16     a.affiche()
17
18     print("Fin du test")
```

```

1 '''
2 Fichier : main1.py
3 '''
4 from module1 import *
5
6 print("Mon programme 1")
7 a = module1(2)
8 a.affiche()

```

Les paquets

Les paquets sont des répertoires contenant un ou plusieurs modules. Pour que Python considère un dossier comme un paquet, il doit contenir un fichier `__init__.py`.

Exemple 2 : Importer un module à l'intérieur d'un paquet

```

1 '''
2 Fichier : module2/module2.py
3 '''
4
5 class module2():
6
7     def __init__(self, couleur):
8         self.couleur = couleur
9
10    def affiche(self):
11        print(self.couleur)
12
13 if __name__ == "__main__":
14     print ("Début du test")
15     a = module2(1)
16     a.affiche()
17
18     print("Fin du test")

```

```

1 '''
2 Fichier : module2/__init__.py
3 '''
4 __all__ = ["module2"]
5
6 from module2.module2 import module2

```

```

1 '''

```

```

2  Fichier : main2.py
3  '''
4  from module2 import module2
5  #from module2 import *
6
7  print("Mon programme 1")
8  a = module2(2)
9  a.affiche()

```

Composition

Exemple 3 : Importer la classe Config et l'utiliser comme attribut d'une classe

```

1  '''
2  Fichier : config/Config.py
3  '''
4  import logging
5
6  class Config():
7      def __init__(self):
8          # Niveau des log
9          self.LOG_LEVEL = logging.DEBUG
10         #self.LOG_LEVEL = logging.CRITICAL
11
12         # E-mail
13         self.EMAIL_USER = "xxx.yyyl@CollegeAhuntsic.qc.ca"

```

```

1  '''
2  Fichier : config/__init__.py
3  '''
4  __all__ = ["Config"]
5  from Config.Config import Config

```

```

1  '''
2  Fichier : main3.py
3
4  Classe principale
5  '''
6  from Config import *
7
8  class Main:
9
10     def __init__(self):
11         self.config = Config()
12
13     def affiche(self):

```

```

14         print("Niveau de log: " + str(self.config.LOG_LEVEL))
15         return 0
16
17     '''
18     Programme principal...
19     '''
20     def main(args):
21         print("La méthode main(): " + str(args))
22         monProg = Main()
23         monProg.affiche()
24         return 0
25
26     if __name__ == '__main__':
27         import sys
28         print("Mon programme principal...")
29         sys.exit(main(sys.argv))

```

Héritage

Exemple 4 : Importer la classe Config et utiliser l'héritage

```

1  '''
2  Fichier : main4.py
3
4  Classe principale
5  '''
6  from Config import *
7
8  class Main(Config):
9
10     def __init__(self):
11         super().__init__()
12
13     def affiche(self):
14         print("Niveau de log: " + str(self.LOG_LEVEL))
15         return 0
16
17     '''
18     Programme principal...
19     '''
20     def main(args):
21         print("La méthode main(): " + str(args))
22         monProg = Main()
23         monProg.affiche()
24         return 0
25
26     if __name__ == '__main__':
27         import sys
28         print("Mon programme principal...")
29         sys.exit(main(sys.argv))

```

Références

- <https://docs.python.org/fr/3.5/tutorial/modules.html>
- https://python.sdv.univ-paris-diderot.fr/14_creation_modules/
- <https://www.pierre-giraud.com/python-apprendre-programmer-cours/module-paquet/>